تشريح طلب API حقيقي

الفصل 3 · Request → Response

الطلب (Request)

```
POST /v1/articles HTTP/1.1
Host: api.site.com
Authorization: Bearer eyJhbGci…
Content-Type: application/json

{"title": "Hi", "draft": true}
```

الرد (Response)

```
HTTP/1.1 201 Created
Location: /v1/articles/42
Content-Type: application/json
RateLimit: "default";r=99;t=60

{"id": "42", "title": "Hi"}
```

أول سطر: Method و Path (أو الـ Status). بعده Headers، وسطر فاضي، وبعدين الـ Body بالـ JSON.

قواعد JSON

الفصل 4 · JSON rules

```
{
  "id": 42,
  "title": "Hi",
  "price": 9.5,
  "tags": ["api", "json"],
  "author": {"name": "Sara"},
  "draft": false,
  "deletedAt": null
}
```

- ف4 ★ الـ Keys و الـ Strings بـ "" دايمًا — `"key"`
- ف4 ⚠ Trailing comma = خطأ — `[1, 2,]`
- ف4 ⚠ مفيش Comments في JSON — `// comment`
- ف5 مش موجودين في JSON — `undefined · NaN · Date`

JSON في JavaScript

الفصل 5 · parse

- ف5 ★ نص ← Object — `JSON.parse(text)`
- ف5 ★ Object ← نص — `JSON.stringify(obj)`
- ف5 منسّق بمسافتين — `JSON.stringify(obj, null, 2)`
- ف5 نسخة عميقة (مش stringify) — `structuredClone(obj)`

JSON Schema

الفصل 7 · العقد

```
{
  "type": "object",
  "required": ["title"],
  "properties": {
    "title": {"type": "string"},
    "tags": {
      "type": "array",
      "items": {"type": "string"}
    }
  },
  "additionalProperties": false
}
```

- ف7 اتحقق من البيانات في JS — `new Ajv2020({ allErrors: true })`

jq في الـ Terminal JSON

الفصل 6

- ف6 ★ حقل واحد — `jq '.title'`
- ف6 ★ أول عنصر · كل العناصر — `jq '.items[0]' · '.items[]'`
- ف6 العدد — `jq '.items | length'`
- ف6 ★ فلترة — `jq 'map(select(.done))'`
- ف6 Object جديد بحقول معيّنة — `jq '{id, title}'`
- ف6 قيمة بديلة — `jq '.x // "default"'`
- ف6 نص خام من غير "" — `jq -r '.[].title'`
- ف6 متغير من الـ Shell — `jq --arg id 42 '.[] | select(.id == $id)'`
- ف6 CSV — `jq -r '.[] | [.id, .title] | @csv'`

fetch في JavaScript

الفصل 10 · fetch

```
const res = await fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hi" }),
  signal: AbortSignal.timeout(10_000),
});
if (!res.ok)
  throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

- ف10 ⚠ 404 و 500 مبترميش لوحدها — `if (!res.ok) throw ...`
- ف10 الـ Body يتقرا مرة واحدة — `await res.json() · res.text()`
- ف10 Header اقرا — `res.headers.get("link")`
- ف10 ★ Query بأمان — `new URLSearchParams({ q, limit: 10 })`

curl للـ APIs

الفصل 8 · curl

- ف8 ★ اطلب واقرا JSON — `curl -s URL | jq`
- ف8 ★ الـ Body + الـ Headers — `curl -i URL`
- ف8 ★ POST بـ JSON — `curl --json '{"a":1}' URL`
- ف12 ابعت توكن — `curl -H 'Authorization: Bearer $T' URL`
- ف8 Body من ملف — `curl -X PATCH -d @p.json URL`
- ف8 Query متشفّر — `curl -G URL --data-urlencode 'q=a b'`
- ف8 Exit 22 لو 4xx/5xx — `curl --fail-with-body URL`
- ف8 الـ Status بس — `curl -w '%{http_code}' -o /dev/null -s URL`

تشريح أمر: curl --json

ف8

```
curl -s --json '{"title":"Hi"}'
https://api.site.com/v1/articles
```

1. الأداة
2. اسكت
3. Body JSON + Content-Type + Accept
4. البيانات
5. الـ Endpoint

HTTPie

الفصل 8 · http

- ف8 ★ GET ملوّن — `http URL`
- ف8 String و JSON values — `http POST URL title=Hi n:=5`
- ف8 Query param — `http URL q==term`
- ف8 Header — `http URL Header:value`

الـ OpenAPI و Docs

الفصل 11 · Spec

- ف11 Path param في الـ Docs — `GET /v1/articles/{id}`
- ف21 افحص الـ Spec — `npx -y @redocly/cli@latest lint openapi.yaml`
- ف21 Types من الـ Spec — `npx openapi-typescript openapi.yaml -o api.d.ts`

APIs تتمرّن عليها

الفصل 9 · مجانية

- ف9 ★ بوستات وهمية: CRUD كامل — `https://jsonplaceholder.typicode.com/posts`
- ف9 منتجات + فلترة — `https://dummyjson.com/products?limit=2`
- ف9 Pagination — `https://pokeapi.co/api/v2/pokemon?limit=3`
- ف9 ★ يرجّعلك طلبك زي ما هو — `https://httpbin.org/anything`
- ف9 الطقس من غير مفتاح — `https://api.open-meteo.com/v1/forecast?…`

الـ Auth — Authentication

ف12 · ف13

ف12 Header في API key
```
x-api-key: KEY
```

ف12 Basic (Base64) مش تشفير!
```
curl -u user:pass URL
```

ف12 ★ الأشهر: توكن
```
Authorization: Bearer <token>
```

ف12 بص جوه الـ JWT
```
cut -d. -f2 <<< $JWT | base64 -d
```

تشريح الـ JWT

ف12

```
1 eyJhbGci…  2 .eyJzdWIi…  3 .SflKxw…
```

1. Header: نوع التوقيع (Base64url)
2. Payload: مين انت وإمتى ينتهي (exp). مقروء لأي حد!
3. Signature: السيرفر بس يقدر يعملها

OAuth 2.0 باختصار — PKCE

الفصل 13

1. ودّي المستخدم لصفحة الدخول — `GET /authorize`
2. يرجع بكود مؤقت — `?code=abc`
3. بدّل الكود بـ access_token — `POST /token`
4. استخدمه في الطلبات — `Bearer access_token`

الـ Webhooks

الفصل 20

ف20 توقيع الطلب
```
X-Hub-Signature-256: sha256=…
```

ف20 احسب التوقيع بنفسك
```
openssl dgst -sha256 -hmac SECRET -hex < body
```

ف20 ⚠ الـ Body الخام للتحقق
```
express.raw({ type: "application/json" })
```

تصميم REST — REST design

ف14 – ف19

ف14 ★ الكل (جمع، مش getArticles)
```
GET /v1/articles
```

ف14 واحد
```
GET /v1/articles/42
```

ف16 اعمل جديد + Location
```
POST /v1/articles → 201
```

ف15 عدّل جزء
```
PATCH /v1/articles/42
```

ف16 امسح، من غير Body
```
DELETE /v1/articles/42 → 204
```

ف18 فلترة وترتيب
```
?status=draft&sort=-createdAt
```

ف18 Cursor بالـ Pagination
```
?limit=20&cursor=abc
```

ف19 بعد 429: استنى
```
Retry-After: 30
```

Express 5 — الجسر للباك إند

الفصل 22

```
app.use(express.json());
app.post("/v1/articles", (req, res) => {
  const a = save(req.body);
  res.status(201)
     .location(`/v1/articles/${a.id}`)
     .json(a);
});
// error handler = 4 arguments
app.use((err, req, res, next) =>
  res.status(500).json({ status: 500 }));
```

شكل الـ Errors — RFC 9457

الفصل 17

```
{
  "type": "/errors/validation",
  "title": "Invalid article",
  "status": 422,
  "detail": "title is required",
  "errors": [
    {"pointer": "/title"}
  ]
}
```

ف17 نوع الرد ده
```
Content-Type: application/problem+json
```

حاجات بتتلخبط بينها — X vs Y

قارن بسرعة

بدل	الفرق	خلي بالك	ف
الشكل			
JSON / JS Object	نص بقواعد صارمة / حاجة في الذاكرة	"" إجباري في JSON	ف5
REST / GraphQL / Webhook	URLs و Query / Methods / واحد / السيرفر يكلمك		ف2
`JSON.parse` / `res.json`	نص عندك / يقرا الـ Body ويحوّله	res.json بيرجع Promise	ف10
الـ Status و Methods			
`PATCH` / `PUT`	استبدل كله / عدّل اللي بعته بس	PUT من غير حقل = اتمسح	ف15
`PUT` / `POST`	مش Idempotent / تكراره نفس النتيجة	Idempotency-Key مع POST	ف15
`422` / `400`	مش مفهوم أصلًا / مفهوم بس البيانات غلط		ف16
`403` / `401`	مين انت؟ / عارفك ومش مسموح		ف16
الأمان والبيانات			
Authentication / Authorization	مين انت / مسموح لك بإيه	401 للأولى · 403 للتانية	ف12
API key / Bearer / OAuth	مفتاح ثابت / توكن بينتهي / دخول باسم المستخدم	الـ Key عمره ما يبقى في الفرونت	ف13
offset / cursor	page=3 بسيط / مكان ثابت ومش بيتلخبط	cursor للبيانات اللي بتتغير	ف18

لو حصل كذا... اعمل كذا — Troubleshooting

مشاكل شائعة

أول خطوة دايمًا: `curl -i URL` وشوف الـ Status والـ Content-Type قبل ما تلوم الكود.

ف5 **Unexpected token '<'** · السيرفر رجّع HTML مش JSON
```
curl -i URL
res.text()
```

ف4 **SyntaxError في JSON** · Trailing comma أو ' بدل "
```
jq . file.json
```

ف8 **curl -d والسيرفر مستلمش حاجة** · نسيت الـ Content-Type
```
curl --json '{...}' URL
```

ف10 **body stream already read** · قريت الـ Body مرتين
```
const data = await res.json()
```

ف10 **404 ومفيش Error اترمى** · fetch مبيرميش على 4xx
```
if (!res.ok) throw ...
```

ف12 **401 Unauthorized** · التوكن ناقص أو خلص
```
curl -H 'Authorization: Bearer $T'
```

ف19 **429 Too Many Requests** · بتطلب كتير
```
Retry-After
backoff
```

ف20 **توقيع الـ Webhook مش مطابق** · حسبته على JSON بعد Parse
```
express.raw(...)
```

ف8 **zsh: no matches found** · الـ ? في الـ URL
```
curl 'URL?q=1&b=2'
```

ف13 **blocked by CORS** · السيرفر لازم يسمح
```
Access-Control-Allow-Origin
```